

1. Prove or disprove that the following language is context-free: $L = \{x2y : x, y \in \{0, 1\}^*, x = y\}$

Example strings: $2 \in L$, 020 , 121 , 101021010

Assume by contradiction that the language is context free

$\exists p$ such that $\forall w \in L$ with $|w| \geq p$, w can be written as $uvxyz$ and satisfies the following conditions:

A) $\forall i \in \mathbb{N}, uv^i xy^i z \in L$

B) $|vy| > 0$

C) $|vxy| \leq p$

Let p be the pumping length.

Let $w = x2x$ which is still valid for the language and $x = 0^p$. Therefore, $w = 0^p 2 0^p$

The length of w is $2p + 1 \geq p$

$w = uvxyz$ and length is $2p+1$, so vxy lies within the first ' p ' characters or includes the 2.

Case 1:

Vxy is completely in the first x portion (0^p)

Pump the p value down with value ' k ', so the first half of the string becomes 0^{p-k}

But now this string does not belong to L , as $0^{p-k} \neq 0^p$. The first half of the string now has less 0's than the second half and therefore violates condition A as $uv^i xy^i z \notin L$ with $i > 0$

Case 2:

Vxy is completely in the second half of the string (0^p)

- Would be the same methodology as case 1, but for the second half of the string. Since the string I chose is $w = x2x$, the second half is identical to the first.

- Would violate condition A as $uv^i xy^i z \notin L$ with $i > 0$

Case 3:

Vxy is completely in middle portion of string ('2')

- V and y would encompass the '2' section, pumping either of the variables would result in either more or less 2's.

- The language only accepts strings with one '2' character

- Therefore this also proves that it would violate condition A as $uv^i xy^i z \notin L$ with $i > 0$

Case 4:

Vxy encompasses first half of string into the '2'

- When pumping with value k , either the x or 2 gets pumped.

- If the x gets pumped with pumping value k , then there will be a mismatch between the number of 0's between the first and second half of the strings, as $x^p \neq x^{p+k}$. Therefore, it would no longer belong to the language.

- If the 2 gets pumped with pumping value k , then the string will also no longer belong to the language, as the language only accepts one '2' and not $1+k$ 2's.

- Hence it would violate condition A as $uv^i xy^i z \notin L$ with $i > 0$

Case 5:

Vxy encompasses from 2 to second half of the string.

- Same logic as case 4, but with the second half of the string instead of the first.
- Also leads to condition A being violated as $uv^i xy^i z \notin L$ with $i > 0$

Case 6:

Vxy encompasses across both halves of the string

- This is not possible, as $|vxy| \leq p$ regulates this from happening. Both halves have length p and the '2' character is at position $p + 1$.

Therefore there is a contradiction as the condition $uv^i xy^i z \in L$ has proven faulty with this particular language. Hence, the language $L = \{x^2 y : x, y \in \{0, 1\}^*, x = y\}$ is NOT context free.

2. Prove or disprove that the language $L = \{w \in \{a, b, c\}^* : \max\{|w|_a, |w|_b, |w|_c\} \leq 2 \min\{|w|_a, |w|_b, |w|_c\}\}$ (where $|w|_x$ indicates the number of occurrences of the symbol x in w) is context-free.

- $\max\{a, b, c\} \leq 2 * \min\{a, b, c\}$
- So no symbol can occur more than twice as often as any other symbol.
- Example strings: $abc, aaabbbccc, aaaaabbbccc$
- Assume that the language is context-free for the sake of contradiction
- $\exists p$ such that $\forall w \in L$ with $|w| \geq p$, w can be written as $w = uvxyz$ and satisfies the following conditions:
 - A) $\forall i \in \mathbb{N}, uv^i xy^i z \in L$
 - B) $|vy| > 0$
 - C) $|vxy| \leq p$

- Choosing string w as an example case where $w = a^p b^p c^{2p}$ and $w \in L$.
- Split the string into $w = uvxyz$ and consider the rules B and C listed above, where $|vy| > 0$ and $|vxy| \leq p$.
- Since the length of vx is p or lower, it only contains the first parts of string w . Need to consider the different cases for vxy .

Case 1: vxy only contains a 's.

- Pumping with value ' k ' will only affect the number of a 's in the string.
- String is now $a^{(p+k)} b^p c^{2p}$ when $k > 0$
- Therefore, $|a| = p+k$, $|b| = p$, and $|c| = 2p$

- This can still allow the string to belong to the language, as if a is assumed to be largest and b is smallest, then with k sizes smaller and equal than p length, $|a| \leq 2 * |b|$.
- However, if the pumping size 'k' is larger than the pumping length p, a will then become too large and will eventually force the string to not belong to the language.
- Then $|a| > 2 * |b|$
- So for some $i > 0$, $uv^ix^iyz \notin L$

Case 2: vxy only contains b's

- Same as case 1 but with b's instead of a's
- Pump with value = k, so string is now $a^p b^{(p+k)} c^{2p}$ when $k > 0$
- Therefore, $|a| = p$, $|b| = p + k$, and $|c| = 2p$
- This can still allow the string to belong to the language, as if b is assumed to be largest and a is smallest, then with k sizes smaller and equal than p length, $|b| \leq 2 * |a|$.
- However, if the pumping size 'k' is larger than the pumping length p, b will then become too large and will eventually force the string to not belong to the language.
- Then $|a| > 2 * |b|$
- So for some $i > 0$, $uv^ix^iyz \notin L$

Case 3: vxy only contains c's

- Same as case 1 and 2 but with c's instead of a's and b's
- Pump with value = k, so string is now $a^p b^p c^{2(p+k)}$ when $k > 0$
- Therefore, $|a| = p$, $|b| = p$, and $|c| = 2(p + k)$ or $2p + 2k$
- With any k value > 0 , c will become too large and will eventually force the string to not belong to the language.
- Then $|c| > 2 * |b|$ or $2 * |a|$
- So for some $i > 0$, $uv^ix^iyz \notin L$

Case 4: vxy spans two different character types (Ex: In the a and b portions)

- For this case, going to pump down both a and b with value 'k'
- So the string is now $w = a^{(p-k)} b^{(p-k)} c^{2p}$
- Therefore, $|a| = p - k$, $|b| = p - k$, and $|c| = 2p$
- With any k value > 0 , the amounts of a and b will become too small and will result in the string not belonging in the language.
- Since $|c| > 2 * |b|$ or $2 * |a|$, the string does not belong in the language.
- So for some $i > 0$, $uv^ix^iyz \notin L$
- This would be the same case for other overlapping cases as well.
- With 'c' and 'b' overlap, pumping up c will result in the a occurrence number being too low.

Therefore there is a contradiction as the condition $uv^ix^iyz \in L$ for some $i > 0$ has proven faulty with this particular language. Hence, the language $L = \{w \in \{a, b, c\}^* : \max\{|w|_a, |w|_b, |w|_c\} \leq 2 \min\{|w|_a, |w|_b, |w|_c\}\}$ is NOT context free.

3I: Explain the behavior of the Turing machine informally and give a compact expression for the language L it accepts.

Informal State Overview:

State S0:

If the input is 0 then write 0 and move right while staying in state s0

If the input is 1, then write 1 and move right while staying in state s0

If the input is #, then go to state s1 and move left.

State S1:

If the input is 0, write # and go to state s2

If the input is 1, write # and go to state s6

If the input is #, go to the halt state

State S2:

If the input is 0, write 0 and stay in state s2

If the input is 1, write 1 and stay in state s2

If the input is #, go to state s3 and move left.

State S3:

If the input is 0, stay in state s3 and move left

If the input is 1, move left and stay in state s3

If the input is #, go to state s4 and move right

State S4:

If the input is 0, write # and go to state s5

If the input is 1, write 1 and stay in state s4

If the input is #, write # and go to state s0

State S5:

If the input is 0, write 0 and stay in state s5

If the input is 1, write 1 and stay in state s5

If the input is #, go to state s0 and move right

State S6:

If the input is 0, write 0 and stay in state s6

If the input is 1, write 1 and stay in state s6

If the input is #, go to state s7 and move left

State S7:

If the input is 0, stay in state s7 and move left.

If the input is 1, stay in state s7 and move left
If the input is #, go to state s8 and move right

State S8:

If the input is 0, write 0 and stay in state s8
If the input is 1, go to state s9 and write #
If the input is #, go to state s0 and write #

State S9:

If the input is 0, write 0 and stay in state s9
If the input is 1, stay in state s9 and write 1
If the input is #, go to state s0 and move right

- In state s0, the machine moves right and skips the inputted 0's and 1's until it finds the first '#'
- In state s1 after the first '#' was inputted, then goes back and marks the last unmatched 0 or 1 with a '#'. If it is a 0 it then goes to state s2, and if it is a 1 then goes to state s6.
- In state s1, if the machine reads in '#' as input on both sides of the '#', it accepts the string as all characters before and after the middle '#' symbol have been matched together.

0-Handling:

- In state s2, the machine moves left until it finds a '#' symbol, and then switches to state s3.
- In state s3, the machine moves right to find the first unmatched 0.
- In state s5, the machine changes the 0 to # and goes back to state s0

1-Handling:

- In state s6, the machine moves left until it finds '#' symbol and then switches to state s7
- In state s7, the machine moves right and finds the first unmatched 1.
- In state s9, the machine changes the 1 to # and goes back to state s0

Informal Behavior Description:

- Starts at the left and moves right until it finds the first '#'
- Then goes into this loop:
 - Moves left until it finds the most near unmatched character before the '#'
 - Then it indicates it is unmatched with the '#' symbol
 - Then it goes right and past the '#' until it finds first unmatched character of same type. Then it marks that character with '#'
- This loop is repeated until all of the symbols are matched before and after the middle '#'
- Accepts the string if all symbols before and after the middle '#' are matched.

Compact Expression: $L = \{x\#y \mid x, y \in \{0,1\}^*, x = y^R\}$

- String is accepted if it is in the format $x \# y$

- $x, y \in \{0,1\}^*$
- $x = y^R$

4II: For a string $x \in L$, give the polynomial order for the number of steps required by M to

accept x , as a function of $n = |x|$

Strings of form $a_1 \dots a_k \# b_1 \dots b_k$ where $x = a_1 \dots a_k$ and $y = b_1 \dots b_k$, $x = y^R$, and the string has a dividing '#' in the middle of it.

Therefore the total input length is $n = 2k + 1 \rightarrow k = (n-1)/2$

Each round matches one pair of characters to each other (one from each side).

of rounds = $k = (n-1)/2$

$(n-1)/2 \in O(n)$

In every individual round:

- Machine scans right to find a '#' (n steps at most)
- Moves left to find the nearest to middle unmatched symbol in x (n steps at most)
- Moves right past the middle '#' to find the nearest to middle unmatched character in y (n steps at most)
- Mark it with a '#' and then return to the middle '#' (n steps at most)

Therefore, the time would be $O(n)$ as the highest amount of memory used was n

$O(n)$ rounds where $O(n)$ steps for each round, so:

$O(n) * O(n) = O(n^2)$

Therefore, $T(n) = O(n^2)$ and the turing machine accepts a string belonging to language L in $O(n^2)$ time.

$\subseteq \wedge \in \forall \exists \neg \Sigma \varepsilon \cap$

4I: Program receives in input a C program x, and counts the number of statements in x.

YES, the program can be written. The number of statements in a program is in the syntax and can be counted through parsing the program. Therefore it is algorithmically possible.

4II: Program receives in input a C program x and an input string w, and counts the number of statements executed at least once when x runs on input w.

- Dynamic analysis $\rightarrow$ execute the program x on input w and then count the statements that were executed at least once.
- Program must halt for inputs x and w and returns number of executed statements.
- Must work for all c programs, so need to consider forever looping programs.

- Considering an infinite loop, the program would never terminate and therefore the statement number would be infinity or practically just uncountable.
- But the program must stop at some point and return the number of statements found after running $x(w)$, therefore cannot always count how many statements were executed as the length of the program's runtime must be known.

Therefore the program CANNOT be written. NO.

4III: Receives in input a C program x and an input string w , and counts the number of statements never executed when x runs on input w .

- This concept must apply to all c programs, including infinite looped programs.
- To count how many statements were never executed, the program would eventually need to halt and presents the halting problem once again. If the program never halts to completion, then the number of nonexecuted statements can never be countable.

Therefore the program CANNOT be written. NO.

4IV: Receives in input a C program x and decides whether x is syntactically correct

- This program does not need to run/simulate the program x . Only needs to check if the program x follows the rule of the C language.
- C has context free grammar and parsing through the program is algorithmically possible.
- Any string can be accepted as a valid syntax in C or rejected as invalid and the decision can be done in a finite amount of time without a program running, which eliminates the case of an infinite loop.

Therefore the program CAN be written. YES

4V: Receives in input two natural numbers and computes a specific function $f: \mathbb{N}^2 \rightarrow \mathbb{N}$

- Asks for a specific function $f: \mathbb{N}^2 \rightarrow \mathbb{N}$, but the bounds for choosing a function can be any function that is possible in the range.
 - So, every function $f: \mathbb{N}^2 \rightarrow \mathbb{N}$ must be computable and able to be run by the program.
 - This set of all functions $f: \mathbb{N}^2 \rightarrow \mathbb{N}$ is uncountably infinite and the set of all computer programs is countable
 - Therefore, there exists some specific functions f from $\mathbb{N}^2$ to $\mathbb{N}$ that no program can compute
- For example:

$f(a,b) = 1$ if the turing machine with code a halts on input b and 0 otherwise.

This function attempts to solve the halting problem and is not computable.

Therefore the program CANNOT be written. NO. There exists functions in the set that cannot be solved by any program or algorithm.

4VI: Receives in input a arithmetic expression e composed of integers and the three arithmetic operators “+”, “-”, and “×”, and computes its value.

- The arithmetic expression e creates a context-free language and can be parsed through

- You can create an algorithm that can parse the expression and each expression will have unique solution.
 - Just need to analyze the operators and numbers, then parse through expression using a grammar, and then use a stack or parse tree for getting the value.
 - Halting problem does not apply here.
 - Extremely large numbers can be computed because there is unlimited memory.
- Therefore the program CAN be written. YES.**